Frozen-Expert Mixture of Experts for Hindi-English Code-Mixed Sequence Labelling

[Priyanshu Narayan, Samarth Agrawal, Satyasheel Singh, Vaishnav Vaijnath Pasarge]
Deep Learning Project
CS F425
BITS Pilani
{f20230283; f20230761; f20230261; f20230631} :@pilani.bits-pilani.ac.in

***Abstract*—Hindi-English code-mixing is pervasive in Indian social media, where speakers fluidly alternate between the two languages within a single utterance. Standard monolingual models struggle with this mixed register. We present a Frozen-Expert Mixture of Experts (FE-MoE) framework that blends two independently pre-trained, completely frozen language models-HingBERT (Hindi-English specialist) and RoBERTa-base (English generalist)-via a lightweight learned router, training only $\sim$200K parameters. We systematically evaluate three router architectures (MLP, BiGRU, CNN) and four ablation axes (temperature, routing mode, granularity, input) across Language Identification (LID) and Part-of-Speech (POS) tagging. The best MoE variant (MLP, τ=0.1) achieves 88.98% weighted F1 on LID, within 0.67 points of a fully fine-tuned HingBERT (89.65%) while using only 0.24% as many trainable parameters. We also contribute interpretability analyses revealing that the router learns linguistically meaningful routing: Hindi tokens receive significantly higher HingBERT weight (α) than English tokens.**

***Index Terms*—code-mixing, mixture of experts, language identification, part-of-speech tagging, Hindi-English, frozen experts, soft routing**

I. Introduction

Hindi-English code-mixing-the practice of alternating between Hindi and English within a single sentence-is ubiquitous in Indian social media, messaging, and informal writing. Computational models for such text face a fundamental challenge: no single pre-trained language model captures both the Hindi morphology and syntax *and* the English semantics equally well.

Prior work has addressed this through code-mixed pre-training [1], multilingual models, and task-specific fine-tuning. However, fine-tuning large models is compute-intensive ($\sim$85M parameters for BERT-scale models), and models fine-tuned on one task tend to forget representations useful for others.

We propose a complementary approach: **freeze both experts** and train only a small router network that learns, for each token, how much to rely on the Hindi-specialist versus the English-generalist. This draws inspiration from Soft MoE [2] and Late Fusion in multimodal learning, but applies it at the level of independently pre-trained, task-agnostic language model encoders.

Our contributions are:

1) A frozen dual-expert MoE framework with three router architectures (MLP, BiGRU, CNN) for code-mixed sequence labelling.
2) Systematic ablations over temperature, routing mode (soft/hard/sentence-level), and router input.
3) Interpretability analyses showing the router learns linguistically valid language-specific routing.
4) Demonstration that 200K trainable parameters match a 85M-parameter fine-tuned model on LID.

II. Related Work

A. Mixture of Experts

The Sparsely-Gated MoE layer [3] introduced the foundational design of routing tokens to expert sub-networks via a learned gating function, enabling 1000$\times$ capacity scaling. A key limitation is training instability and token dropping when routing is discrete. Puigcerver et al. [2] addressed this with *Soft MoE*, which replaces hard assignment with a fully-differentiable weighted combination of expert outputs-the same principle underlying our soft routing formulation. MoEBERT [4] adapts pre-trained BERT into an MoE structure via importance-guided distillation, achieving inference speedup while preserving knowledge. Unlike MoEBERT, we do not modify the expert internals; instead we keep both experts entirely frozen and train only the routing mechanism. MoE-LPR [5] upcycles a dense LLM into MoE by freezing original FFN weights and adding expert copies with a language-prior router-closest in spirit to our method, but applied to monolingual LLM expansion rather than cross-lingual code-mixing.

B. Code-Mixed NLP

HingBERT [1] is a BERT-base model pre-trained on 52.93M Hindi-English code-mixed tweets, providing strong Hindi-English representations. We use it as our Hindi-specialist expert. BERT [6] and RoBERTa [7] serve as the English-generalist backbone; we use RoBERTa-base as the second expert. The LinCE benchmark [8] established standardised evaluation for code-switching across LID, NER, and POS tasks. We evaluate on COMI-LINGUA [9], an expert-annotated large-scale dataset covering Hindi-English LID and POS with high inter-annotator agreement ($\kappa \geq 0.81$).

C. Language-Aware Routing

LR-MoE [10] proposes frame-wise language routing via a shared language identification network to guide MoE expert selection in code-switched speech recognition. While our setting is text rather than speech, the principle of routing based on language identity is directly related. Our router learns to route *implicitly* from hidden states rather than requiring an explicit LID signal, and we show that it recovers language-level routing without supervision on α.

III. Methodology

A. Model Architecture

Given a sequence of N words, each word w_i is independently tokenized by two tokenizers: HingBERT's WordPiece and RoBERTa's BPE. Because these tokenizers produce different sub-token counts for the same word, we apply **sub-token alignment**: the hidden states of all sub-tokens belonging to the same word are averaged, yielding consistent word-level representations $\mathbf{h}_i^{\text{hing}}, \mathbf{h}_i^{\text{rob}} \in \mathbb{R}^{768}$ for each word i.

Both experts are kept **completely frozen** throughout training. A learned router receives these representations and produces a scalar blending weight $\alpha_i \in (0, 1)$ per word:

$$\hat{\mathbf{h}}_i = \alpha_i \cdot \mathbf{h}_i^{\text{hing}} + (1 - \alpha_i) \cdot \mathbf{h}_i^{\text{rob}} \tag{1}$$

The blended representation $\hat{\mathbf{h}}_i$ is passed to a linear task head for per-token label prediction.

B. Router Architectures

We implement three routers, all consuming the concatenated or individual expert hidden states:

RouterMLP (R1–R7): Two linear layers with GELU activation, followed by sigmoid with temperature τ:

$$\alpha_i = \sigma\left(\frac{\text{Linear}(\text{GELU}(\text{Linear}([\mathbf{h}_i^{\text{hing}}; \mathbf{h}_i^{\text{rob}}]))))}{\tau}\right) \tag{2}$$

Input dimensionality is $D = 1536$ (concatenation) or $D = 768$ (single-expert ablations).

RouterGRU (R8): A bidirectional GRU with hidden size 128 processes the sequence of concatenated states, capturing sequential context across positions before projecting to a scalar α.

RouterCNN (R9): A 1D convolution with kernel size $k = 5$ and 128 output channels captures local neighbourhood context (5-token receptive field) before projecting to α.

C. Routing Modes

Beyond the standard token-level soft routing described above, we explore:

- **Hard routing** (R7): Straight-through estimator [11]-binary gate in the forward pass, soft gradient in the backward pass.
- **Sentence-level routing** (R6): α is computed once from the CLS tokens of both experts and broadcast to all token positions.

TABLE I
Main results (mean ± std, two seeds). LID: weighted F1; POS: accuracy.

ID	Description	LID	POS
B1	HingBERT fine-tuned	**89.65** ± 0.23	**91.09** ± 0.11
B2	HingBERT frozen	80.55 ± 0.06	65.23 ± 0.25
B3	RoBERTa frozen	81.48 ± 0.01	63.18 ± 0.31
B4	Fixed 50/50 avg	80.41 ± 0.31	61.10 ± 0.12
R1	MoE MLP τ=1.0	84.37 ± 3.21	67.76 ± 1.50
R2	MoE MLP τ=0.1	<u>88.98</u> ± 0.65	66.99 ± 1.54
R7	MoE MLP hard routing	87.58 ± 0.10	65.20 ± 0.22
R8	MoE BiGRU router	86.61 ± 0.63	64.22 ± 2.17
R9	MoE CNN (k=5)	88.29 ± 0.12	66.78 ± 0.01

IV. Experimental Setup

A. Tasks and Datasets

We evaluate on two tasks from the COMI-LINGUA corpus [9]:

- **LID**: Per-token language identification into {hi, en, ot}. Metric: weighted F1.
- **POS**: 14-class universal part-of-speech tagging. Metric: token accuracy.

Devanagari-script sentences are filtered from COMI-LINGUA to retain only romanized code-mixed text. Train/val/test splits follow the dataset's native partition with 10% of train carved for validation (seed-controlled).

B. Baselines

- **B1**: HingBERT fully fine-tuned (85M trainable params). Upper bound.
- **B2**: Frozen HingBERT + linear head.
- **B3**: Frozen RoBERTa + linear head.
- **B4**: Fixed 50/50 average of both experts (no router).

C. Training Details

All models use AdamW with lr $= 2 \times 10^{-4}$, linear warmup over 10% of steps, gradient clipping at 1.0, batch size 32, max sequence length 128, and early stopping with patience 3 (max 20 epochs). Results are reported as mean ± std over seeds {42, 123}.

V. Results and Ablations

A. Main Results

Table I shows the main results. The best MoE model (R2, τ=0.1) achieves 88.98% LID F1, within 0.67 points of fully fine-tuned B1 (89.65%), while training only ∼200K parameters versus ∼85M for B1-a 400× reduction. All MoE variants substantially outperform frozen single-expert baselines (B2: 80.55%, B3: 81.48%) and the fixed blend (B4: 80.41%), confirming the value of learned routing.

For POS, MoE models achieve ∼67% accuracy compared to B1's 91.09%. Frozen expert representations lack the task-specific geometric structure needed for POS tagging, and the router cannot compensate without expert adaptation.

TABLE II
ABLATION RESULTS ON LID (MEAN ± STD). ALL MLP, τ=1.0.

Ablation	Config	LID	Δ vs R1
Router input	A2: HingBERT only	86.06 ± 0.24	+1.69
	A3: RoBERTa only	87.42 ± 0.08	+3.05
	R1: Both (full)	84.37 ± 3.21	–
Granularity	R7: Hard token	87.58 ± 0.10	+3.21
	R6: Sentence-lvl	81.20 ± 0.04	−3.17
	R1: Soft token	84.37 ± 3.21	–

B. Temperature Ablation

Table II (top block) shows the effect of sigmoid temperature τ on LID performance. Lower temperatures produce sharper, near-binary routing (τ=0.1: best LID at 88.98%) while higher temperatures collapse toward uniform blending. R1 and R5 (τ=$1.0, 2.0$) exhibit high variance (±3.2%), suggesting near-uniform routing is unstable and seed-sensitive.

C. Router Architecture Ablation

CNN (R9, 88.29%) closely matches MLP (R2, 88.98% at optimal τ), while BiGRU (R8, 86.61%) underperforms on LID. The CNN's local 5-token receptive field appears sufficient for code-mixing, where language alternates at the word level. The BiGRU's sequential nature may cause error propagation across language boundaries.

D. Router Input Ablation

Interestingly, using only one expert's hidden states as router input (A2, A3) outperforms the full concatenation (R1) while reducing variance. RoBERTa-only routing (A3: 87.42%) outperforms HingBERT-only (A2: 86.06%), suggesting RoBERTa's representations carry more discriminative signal for routing even though HingBERT is the stronger expert for Hindi tokens.

Sentence-level routing (R6: 81.20%) performs significantly worse than token-level, confirming that code-mixing requires per-token routing granularity since language identity alternates at the word level.

E. Interpretability: What Does the Router Learn?

Analysis of α distributions on the LID test set reveals clear language-level routing. Hindi tokens (`hi`) receive significantly higher α (higher HingBERT weight) than English tokens (`en`), with `ot` (Other) tokens falling between. This structure emerges without any direct supervision on α-the router learns language-specific routing purely from the task loss. Router α values also correlate with expert hidden-state disagreement: tokens where $\|\mathbf{h}^{\text{hing}} - \mathbf{h}^{\text{rob}}\|_2$ is large are routed more decisively (sharper α), while tokens both models agree on show near-uniform blending.

VI. FAILURE CASES AND LIMITATIONS

POS tagging: Frozen expert representations are not adapted for POS, resulting in ∼67% accuracy versus B1's 91%. The router cannot compensate for inadequate task-specific geometry in the frozen hidden states. Partially unfreezing the top transformer layers would likely close this gap.

High variance at τ=1.0: R1 and R5 show ±3.2% variance across seeds, indicating the routing landscape at moderate temperatures is non-convex and sensitive to initialisation. Lower temperatures (R2, τ=0.1) stabilize training significantly (±0.65%).

NER: All frozen-expert models achieve near-zero entity-level F1, consistent with the finding that boundary detection requires expert adaptation. NER is excluded from the main sweep.

Compute: B1 checkpoints (418 MB) cannot be version-controlled in standard git repositories due to size constraints, limiting reproducibility without LFS or cloud storage.

VII. CONCLUSION

We presented FE-MoE, a frozen dual-expert MoE framework for Hindi-English code-mixed sequence labelling. By keeping two complementary pre-trained experts frozen and training only a lightweight router, we achieve 88.98% weighted F1 on LID (within 0.67 points of full fine-tuning) using just 200K trainable parameters. Across three router architectures and four ablation axes, we find that: (i) lower temperature sharpens routing and improves both performance and stability; (ii) CNN-based routing matches MLP at lower sequential cost; (iii) token-level granularity is essential for code-mixing; and (iv) the router learns implicit language-aware routing without explicit LID supervision. The FE-MoE framework offers a practical, parameter-efficient alternative to full fine-tuning for code-mixed NLP, and the interpretability analyses provide insight into how language specialisation emerges from task-driven training.

REFERENCES

[1] R. Nayak and R. Joshi, "L3Cube-HingCorpus and HingBERT: A code mixed hindi-english dataset and BERT language models," in *Proceedings of the WILDRE-6 Workshop at LREC*, 2022. [Online]. Available: https://arxiv.org/abs/2204.08398

[2] J. Puigcerver, C. Riquelme, B. Mustafa, and N. Houlsby, "From sparse to soft mixtures of experts," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2024. [Online]. Available: https://arxiv.org/abs/2308.00951

[3] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean, "Outrageously large neural networks: The sparsely-gated mixture-of-experts layer," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2017. [Online]. Available: https://arxiv.org/abs/1701.06538

[4] S. Zuo, Q. Zhang, C. Liang, P. He, T. Zhao, and W. Chen, "MoEBERT: from BERT to mixture-of-experts via importance-guided adaptation," in *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, 2022, pp. 1610–1623.

[5] H. Zhou, Z. Wang, S. Huang, X. Huang, X. Han, J. Feng, C. Deng, W. Luo, and J. Chen, "MoE-LPR: Multilingual extension of large language models through mixture-of-experts with language priors routing," in *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, 2025. [Online]. Available: https://arxiv.org/abs/2408.11396

[6] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, 2019, pp. 4171–4186.

[7] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, "RoBERTa: A robustly optimized BERT pretraining approach," *arXiv preprint arXiv:1907.11692*, 2019.

[8] G. Aguilar, S. Kar, and T. Solorio, "LinCE: A centralized benchmark for linguistic code-switching evaluation," in *Proceedings of the 12th Language Resources and Evaluation Conference (LREC)*, 2020, pp. 1803–1813.

[9] R. Sheth, H. Beniwal, and M. Singh, "COMI-LINGUA: Expert annotated large-scale dataset for multitask NLP in hindi-english code-mixing," in *Findings of the Association for Computational Linguistics: EMNLP*, 2024. [Online]. Available: https://arxiv.org/abs/2503.21670

[10] W. Wang, G. Ma, Y. Li, and B. Du, "Language-routing mixture of experts for multilingual and code-switching speech recognition," in *Proceedings of Interspeech*, 2023. [Online]. Available: https://arxiv.org/abs/2307.05956

[11] Y. Bengio, N. Léonard, and A. Courville, "Estimating or propagating gradients through stochastic neurons for conditional computation," *arXiv preprint arXiv:1308.3432*, 2013.